

UNIVERSIDADE VEIGA DE ALMEIDA — UVA
ENGENHARIA DA COMPUTAÇÃO

FITLIFE — SISTEMA WEB DE GESTÃO DE ACADEMIA

APLICAÇÃO PARA GESTÃO DE ALUNOS, TURMAS, PROFESSORES, UNIDADES E PAGAMENTOS

ANA CLARA ALVES TORRES – 1240202029

CAIO CÉSAR DE OLIVEIRA MOREIRA - 1240113503

DANIEL DANTAS DUARTE – 1240109771

DIEGO CÉSAR DA COSTA E SILVA GONÇALVES - 1240110085

GABRIEL TEIXEIRA MARTINHO – 1240112994

LETÍCIA PEREIRA MADEIRA – 1240202251

RUAN HILARIO SOARES – 1240109865

RYAN DA COSTA FIGUEIRA – 1240115874

YOHANN MARCELLO PEREIRA SARMENTO – 1240110613

**MARIA LUIZA DE OLIVEIRA DE ARAUJO MORAIS -
1250202437**

RIO DE JANEIRO

2026

ANA CLARA ALVES TORRES – 1240202029
CAIO CÉSAR DE OLIVEIRA MOREIRA - 1240113503
DANIEL DANTAS DUARTE – 1240109771
DIEGO CÉSAR DA COSTA E SILVA GONÇALVES - 1240110085
GABRIEL TEIXEIRA MARTINHO – 1240112994
LETÍCIA PEREIRA MADEIRA – 1240202251
RUAN HILARIO SOARES – 1240109865
RYAN DA COSTA FIGUEIRA – 1240115874
YOHANN MARCELLO PEREIRA SARMENTO – 1240110613
MARIA LUIZA DE OLIVEIRA DE ARAUJO MORAIS -
1250202437

Trabalho acadêmico apresentado à
Universidade Veiga de Almeida como requisito
parcial para avaliação na disciplina de
Laboratório de Desenvolvimento de Software,
do curso de Engenharia da Computação.

Orientador: Prof. Paulo Andrade

RIO DE JANEIRO

2026

RESUMO

Este trabalho apresenta o desenvolvimento do FitLife, um sistema web de gestão de academia concebido na disciplina de Laboratório de Desenvolvimento de Software. O problema tratado é a informalidade e a fragmentação com que muitas academias de pequeno e médio porte administram cadastros de alunos, turmas, contratos e cobranças, frequentemente apoiadas em planilhas e controles manuais que geram retrabalho e perda de informação. O objetivo do projeto foi construir uma aplicação centralizada que reúna, em um único ambiente, o cadastro de alunos com suas restrições médicas, a organização de turmas por modalidade e nível de dificuldade, o quadro de professores, as unidades da rede e o controle financeiro de pagamentos, com destaque para as cobranças em atraso. O método adotado combinou a análise de um estudo de caso do domínio de academias, a modelagem de um banco de dados relacional e o desenvolvimento incremental da aplicação, organizado em etapas e versionado no GitHub. O sistema foi implementado com o framework Django, seguindo uma arquitetura em camadas que separa apresentação, regras de negócio e acesso a dados, e conta com um painel gerencial que consolida indicadores da operação. Como resultado, obteve-se uma aplicação funcional que executa as operações de criação, leitura, edição e exclusão sobre as principais entidades do domínio, além de fluxos específicos como a inscrição de alunos em turmas e o acompanhamento de pagamentos por situação. Conclui-se que a solução atende ao escopo proposto e evidencia o domínio da equipe sobre modelagem de dados, arquitetura de software, desenvolvimento web e boas práticas de versionamento.

Palavras-Chave: Gestão de academia. Desenvolvimento web. Django. Banco de dados relacional. Sistema CRUD.

LISTA DE ILUSTRAÇÕES

Figura 1: Diagrama Entidade-Relacionamento (DER) do sistema FitLife

Figura 2: Tela de Dashboard (painel gerencial)

Figura 3: Tela de Login (acesso ao sistema)

Figura 4: Tela de Listagem de Alunos

Figura 5: Tela de Cadastro de Aluno

Figura 6: Tela de Detalhes do Aluno

Figura 7: Tela de Edição de Aluno

Figura 8: Tela de Confirmação de Exclusão de Aluno

Figura 9: Tela de Listagem de Turmas

Figura 10: Tela de Cadastro de Turma

Figura 11: Tela de Detalhes da Turma

Figura 12: Tela de Edição de Turma

Figura 13: Tela de Inscrição de Aluno em Turma

Figura 14: Tela de Listagem de Inscrições

Figura 15: Tela de Listagem de Professores

Figura 16: Tela de Listagem de Unidades

Figura 17: Tela de Listagem de Pagamentos

Figura 18: Tela de Cadastro de Pagamento

Figura 19: Tela de Detalhes do Pagamento

Figura 20: Tela de Edição de Pagamento

SUMÁRIO

1 INTRODUÇÃO	6
2 FUNDAMENTAÇÃO TEÓRICA.....	8
2.1 APLICAÇÕES WEB	8
2.2 O FRAMEWORK DJANGO.....	8
2.3 BANCO DE DADOS RELACIONAL E ORM	8
2.4 ARQUITETURA EM CAMADAS E CAMADA DE SERVIÇOS	9
2.5 CONTROLE DE VERSÃO COM GIT E GITHUB.....	9
2.6 METODOLOGIAS ÁGEIS	9
3 DESENVOLVIMENTO DO SISTEMA.....	10
3.1 ARQUITETURA DA APLICAÇÃO	10
3.1.1 Tecnologias Utilizadas	10
3.2 BANCO DE DADOS	11
3.3 BACKEND	13
3.3.1 Regras de Negócio e Validações	14
3.3.2 Autenticação e Perfis de Acesso	14
3.4 FRONTEND	14
3.5 API REST	14
4 DOCUMENTAÇÃO DAS TELAS.....	15
4.1 TELA DE DASHBOARD	15
4.1.1 Dashboard (Painel Gerencial)	15
4.2 AUTENTICAÇÃO	16
4.2.1 Tela de Login	17
4.3 MÓDULO DE ALUNOS	17
4.3.1 Listagem de Alunos.....	17
4.3.2 Cadastro de Aluno	18
4.3.3 Detalhes do Aluno	19
4.3.4 Edição de Aluno	20
4.3.5 Exclusão de Aluno	21
4.4 MÓDULO DE TURMAS	22
4.4.1 Listagem de Turmas	22
4.4.2 Cadastro de Turma	23
4.4.3 Detalhes da Turma	24

4.4.4 Edição de Turma	25
4.4.5 Inscrição de Aluno em Turma	26
4.5 MÓDULO DE INSCRIÇÕES	27
4.5.1 Listagem de Inscrições	27
4.6 MÓDULO DE PROFESSORES.....	28
4.6.1 Listagem de Professores	28
4.7 MÓDULO DE UNIDADES	29
4.7.1 Listagem de Unidades	29
4.8 MÓDULO DE PAGAMENTOS	30
4.8.1 Listagem de Pagamentos	30
4.8.2 Cadastro de Pagamento	32
4.8.3 Detalhes do Pagamento	32
4.8.4 Edição de Pagamento	33
5 ORGANIZAÇÃO ÁGIL E BACKLOG	35
6 VERSIONAMENTO E GITHUB	36
7 RESULTADOS	37
8 CONCLUSÃO.....	38
REFERÊNCIAS.....	39

1 INTRODUÇÃO

A gestão de uma academia envolve, diariamente, o cruzamento de informações de naturezas distintas: dados cadastrais de alunos, contratos e planos, agenda de turmas, corpo docente, unidades físicas e o controle financeiro das mensalidades. Em muitos estabelecimentos de pequeno e médio porte, esse conjunto de dados é administrado de forma dispersa, apoiado em planilhas e anotações manuais, o que favorece inconsistências, dificulta a busca por informações e compromete a tomada de decisão pela gerência.

Diante desse cenário, este trabalho tem como tema o desenvolvimento de um sistema web capaz de centralizar a operação administrativa de uma academia. O recorte adotado concentra-se nas funcionalidades essenciais de cadastro e acompanhamento — alunos, turmas, professores, unidades, contratos e pagamentos — bem como em um painel gerencial que oferece uma visão consolidada do negócio. A aplicação recebeu o nome de FitLife e foi desenvolvida a partir de um estudo de caso que descreve as regras de negócio de uma rede fictícia de academias.

O objetivo geral do projeto é desenvolver uma aplicação web funcional para a gestão administrativa de uma academia, integrando cadastro de pessoas, organização de turmas e controle financeiro em um único ambiente. Como objetivos específicos, destacam-se: modelar um banco de dados relacional aderente ao domínio; implementar as operações de criação, leitura, edição e exclusão sobre as principais entidades; construir fluxos de negócio específicos, como a inscrição de alunos em turmas respeitando a capacidade máxima; e disponibilizar um painel com indicadores da operação, incluindo pagamentos em atraso e turmas lotadas.

A relevância do tema está na demanda concreta por soluções acessíveis de informatização para academias, que reduzam o retrabalho, aumentem a confiabilidade das informações e apoiem a gestão. Do ponto de vista acadêmico, o projeto permite exercitar, de maneira integrada, os conhecimentos de modelagem de dados, arquitetura de software, desenvolvimento web e versionamento de código, articulando teoria e prática.

Este documento está organizado da seguinte forma. O Capítulo 2 apresenta a fundamentação teórica, revisando os conceitos e tecnologias que sustentam a solução. O Capítulo 3 descreve o desenvolvimento do sistema, abrangendo arquitetura, banco de dados, backend e frontend. O Capítulo 4 documenta as telas da aplicação. O Capítulo 5 trata da organização ágil e do backlog do projeto. O Capítulo 6 aborda o versionamento e o repositório no GitHub. O Capítulo 7 apresenta os resultados obtidos e, por fim, o Capítulo 8 traz as conclusões e as perspectivas de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os principais conceitos e tecnologias que embasam o desenvolvimento do FitLife. São abordados os fundamentos de aplicações web, o framework utilizado, a organização de dados em bancos relacionais, o padrão arquitetural adotado e as práticas de versionamento e desenvolvimento ágil que orientaram o processo de construção do sistema.

2.1 APLICAÇÕES WEB

Uma aplicação web é um software executado em um servidor e acessado pelo usuário por meio de um navegador, dispensando instalação local. A comunicação ocorre sobre o protocolo HTTP, no qual o cliente envia requisições e recebe respostas, normalmente em HTML, complementadas por folhas de estilo (CSS) e scripts (JavaScript). Esse modelo favorece a centralização dos dados, o acesso simultâneo por diferentes perfis de usuário e a manutenção facilitada, uma vez que as atualizações são aplicadas no servidor e imediatamente refletidas para todos os usuários.

2.2 O FRAMEWORK DJANGO

O Django é um framework web de alto nível, escrito na linguagem Python, que promove o desenvolvimento rápido e organizado de aplicações. Ele adota o padrão arquitetural MTV (Model-Template-View), variação do clássico MVC, no qual os models representam as entidades e o acesso ao banco de dados, os templates cuidam da apresentação e as views concentram a lógica que conecta ambos. Entre seus recursos, destacam-se o mapeamento objeto-relacional (ORM), que permite manipular o banco de dados por meio de classes Python; o sistema de rotas (URLs); os formulários com validação integrada; e uma interface administrativa gerada automaticamente, útil para operações de manutenção.

2.3 BANCO DE DADOS RELACIONAL E ORM

O modelo relacional organiza os dados em tabelas compostas por linhas e colunas, nas quais cada tabela representa uma entidade do domínio e os relacionamentos entre entidades são estabelecidos por chaves primárias e estrangeiras. Esse modelo garante integridade referencial e reduz a redundância de informações. No FitLife, o acesso ao banco de dados é intermediado pelo ORM do Django, o que permite descrever as tabelas como classes e realizar consultas de forma programática, sem a escrita manual de comandos SQL na maior parte das operações. Em

ambiente de desenvolvimento, o projeto utiliza o SQLite, banco de dados leve e integrado ao Django, que pode ser substituído por um sistema mais robusto em produção.

2.4 ARQUITETURA EM CAMADAS E CAMADA DE SERVIÇOS

A separação de responsabilidades é um princípio central da engenharia de software. No projeto, além da divisão natural entre apresentação, lógica e dados oferecida pelo padrão MTV, adotou-se uma camada de serviços responsável por concentrar as regras de negócio mais complexas, mantendo as views enxutas e focadas no tratamento das requisições. Essa organização melhora a legibilidade, facilita a manutenção e favorece o reaproveitamento de código, além de tornar as regras de negócio mais fáceis de testar de forma isolada.

2.5 CONTROLE DE VERSÃO COM GIT E GITHUB

O Git é um sistema de controle de versão distribuído que registra o histórico de alterações do código-fonte, permitindo trabalho colaborativo, criação de ramificações e recuperação de versões anteriores. O GitHub é uma plataforma que hospeda repositórios Git e agrega recursos de colaboração, como controle de tarefas e documentação. No desenvolvimento do FitLife, adotou-se uma convenção de mensagens de commit baseada em prefixos semânticos — como feat para novas funcionalidades, fix para correções e refactor para reorganizações —, o que torna o histórico mais legível e rastreável.

2.6 METODOLOGIAS ÁGEIS

As metodologias ágeis propõem o desenvolvimento incremental e iterativo do software, com entregas frequentes e ajustes contínuos a partir do feedback. Conceitos como backlog de produto, priorização de tarefas e ciclos curtos de desenvolvimento (sprints) orientaram a organização do trabalho da equipe. O acompanhamento das atividades foi apoiado por uma ferramenta de gestão de tarefas, permitindo visualizar o que estava planejado, em andamento e concluído ao longo do projeto.

3 DESENVOLVIMENTO DO SISTEMA

Este capítulo descreve como o sistema FitLife foi construído. São apresentadas a arquitetura da aplicação, a modelagem do banco de dados com o respectivo diagrama entidade-relacionamento, a estrutura do backend com suas regras de negócio e a organização do frontend.

3.1 ARQUITETURA DA APLICAÇÃO

A aplicação segue uma arquitetura em camadas, na qual cada requisição percorre um fluxo bem definido. A partir da interação do usuário no navegador, a requisição é recebida por uma view, que aciona a camada de serviços quando há regra de negócio envolvida; esta, por sua vez, opera sobre os models, responsáveis pela persistência no banco de dados. A resposta retorna à view, que renderiza o template correspondente e o devolve ao navegador. O fluxo pode ser resumido pela sequência a seguir.

Frontend (Templates) → Views → Services → Models → Banco de Dados

O código é organizado em aplicações (apps) do Django, cada uma responsável por um contexto do domínio — como alunos, turmas, professores, unidades e pagamentos —, o que mantém o projeto modular e de fácil evolução. Os arquivos estáticos (CSS e imagens) e os templates HTML ficam separados da lógica, reforçando a divisão entre apresentação e regras de negócio.

3.1.1 Tecnologias Utilizadas

O quadro a seguir resume as tecnologias empregadas em cada camada do sistema.

Camada / Finalidade	Tecnologia
Linguagem de programação	Python 3
Framework backend	Django
Banco de dados (desenvolvimento)	SQLite (via ORM do Django)
Camada de apresentação	HTML5, CSS3 e Django Templates
Interatividade no cliente	JavaScript
Controle de versão	Git e GitHub
Gestão de tarefas (ágil)	GitHub Projects / Trello

3.2 BANCO DE DADOS

A modelagem do banco de dados partiu do estudo de caso da rede FitLife, que descreve as entidades do domínio e suas relações. Foram identificadas como entidades principais: Aluno, Plano, Contrato, Pagamento, Modalidade, Turma, Professor, Unidade e Inscrição. O diagrama entidade-relacionamento simplificado é apresentado na figura a seguir.

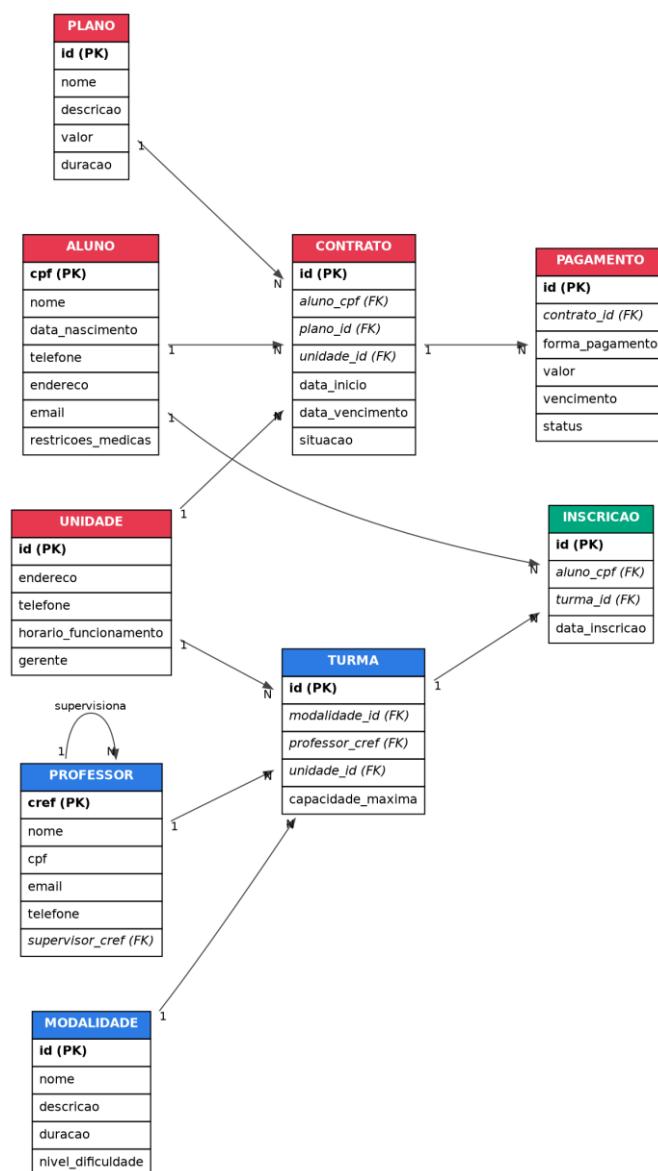


Figura 1: Diagrama Entidade-Relacionamento (DER) do sistema FitLife

Fonte: Elaborado pelos autores (2026).

O relacionamento entre as entidades reflete as regras de negócio do domínio. Um aluno é identificado de forma única pelo seu CPF e pode possuir um ou mais contratos ao longo do tempo; cada contrato vincula-se a um plano e a uma unidade e origina os pagamentos

correspondentes. As turmas são formadas a partir de modalidades, possuem um professor responsável e ocorrem em uma unidade, respeitando uma capacidade máxima de alunos. A participação de um aluno em uma turma é registrada por meio de uma inscrição, caracterizando um relacionamento do tipo muitos-para-muitos entre alunos e turmas. Os professores, identificados pelo número do CREF, podem ainda supervisionar outros professores, o que dá origem a um relacionamento reflexivo na entidade Professor. O quadro seguinte descreve resumidamente as tabelas do modelo.

Entidade	Descrição resumida
Aluno	Pessoa matriculada na academia. Guarda CPF (identificador único), nome, data de nascimento, telefone, endereço, e-mail e restrições médicas.
Plano	Modalidade de contratação oferecida pela academia, com nome, descrição, valor e duração.
Contrato	Vínculo entre aluno, plano e unidade, com datas de início e vencimento e situação (ativo, cancelado ou expirado).
Pagamento	Cobrança gerada a partir de um contrato, com forma de pagamento, valor, vencimento e status (pago, pendente ou atrasado).
Modalidade	Tipo de aula coletiva (dança, jump, luta etc.), com descrição, duração e nível de dificuldade.
Turma	Grupo formado a partir de uma modalidade, com professor responsável, unidade e capacidade máxima de alunos.
Professor	Docente identificado pelo CREF, com nome, CPF, e-mail, telefone e possível supervisor.
Unidade	Filial da rede, com endereço, telefone, horário de funcionamento e gerente responsável.
Inscrição	Registro que associa um aluno a uma turma, controlando a ocupação das vagas.

3.3 BACKEND

O backend concentra a definição das entidades, as regras de negócio e o tratamento das requisições. Os models representam cada tabela do banco como uma classe, declarando os campos, os tipos de dados e os relacionamentos por meio de chaves estrangeiras. As views recebem as requisições, coordenam a validação dos dados e acionam a camada de serviços quando necessário, retornando os templates renderizados ou redirecionando o usuário conforme o resultado da operação.

3.3.1 Regras de Negócio e Validações

Entre as regras de negócio implementadas, destacam-se o controle de vagas nas turmas, que impede novas inscrições quando a capacidade máxima é atingida — sinalizando a turma como lotada —, e o controle de situação dos pagamentos, que identifica cobranças pendentes e atrasadas para exibição no painel gerencial. As validações de formulário garantem o preenchimento dos campos obrigatórios, como CPF, nome e e-mail no cadastro de alunos, e a coerência dos dados informados, prevenindo o registro de informações inconsistentes.

3.3.2 Autenticação e Perfis de Acesso

O sistema conta com um mecanismo de autenticação que exige a identificação do usuário para acesso às funcionalidades administrativas. O perfil gerencial, visível no cabeçalho da aplicação, tem acesso completo às operações de cadastro, edição e exclusão. A interface administrativa nativa do Django, acessível pelo item Admin, complementa a gestão dos dados em situações que exigem manutenção direta sobre os registros. No projeto tem 3 grupos (Administração, Professor e Financeiro). Administração tem acesso a tudo, Professor só tem acesso aos alunos, turmas e inscrições e Financeiro só tem acesso aos pagamentos. Sendo que um perfil pertence a um desses grupos e em as permissões do respectivo grupo

3.4 FRONTEND

A camada de apresentação foi construída com templates HTML integrados ao Django, estilizados com CSS para oferecer uma interface limpa e consistente. A navegação principal é organizada em um menu lateral fixo, dividido nos grupos Principal (Alunos, Inscrições e Pagamentos) e Academia (Turmas, Professores e Unidades), o que facilita o deslocamento entre os módulos. As páginas seguem um padrão visual único, com cabeçalho de título, botão de retorno e cartões que agrupam as informações. Listagens contam com campos de busca, filtros e ordenação, enquanto os formulários destacam os campos obrigatórios e apresentam mensagens de apoio ao preenchimento.

3.5 API REST

O FitLife foi implementado como uma aplicação com renderização no servidor, na qual as páginas são geradas diretamente pelo Django e entregues ao navegador. Dessa forma, o projeto não expõe uma API REST pública como interface principal de comunicação, dispensando a documentação de endpoints, métodos HTTP e payloads. Caso, em evoluções futuras, seja incorporada uma API — por exemplo, para integração com um aplicativo móvel —, esta seção deverá ser complementada com a descrição dos respectivos endpoints.

4 DOCUMENTAÇÃO DAS TELAS

Este capítulo documenta as telas da aplicação. Para cada tela são apresentados o nome, o objetivo, uma descrição resumida e a respectiva captura. As telas cujas capturas ainda não foram inseridas estão indicadas por um espaço reservado, a ser preenchido pela equipe.

4.1 TELA DE DASHBOARD

4.1.1 Dashboard (Painel Gerencial)

Objetivo: Oferecer à gerência uma visão consolidada e imediata da operação da academia.

Descrição: Exibe cartões com os totais de alunos, turmas, professores e unidades, além de indicadores operacionais como pagamentos pendentes, pagamentos atrasados, turmas lotadas e inscrições ativas. Apresenta ainda atalhos rápidos para as ações mais frequentes, como cadastrar novo aluno, nova turma, novo pagamento e visualizar cobranças em atraso.

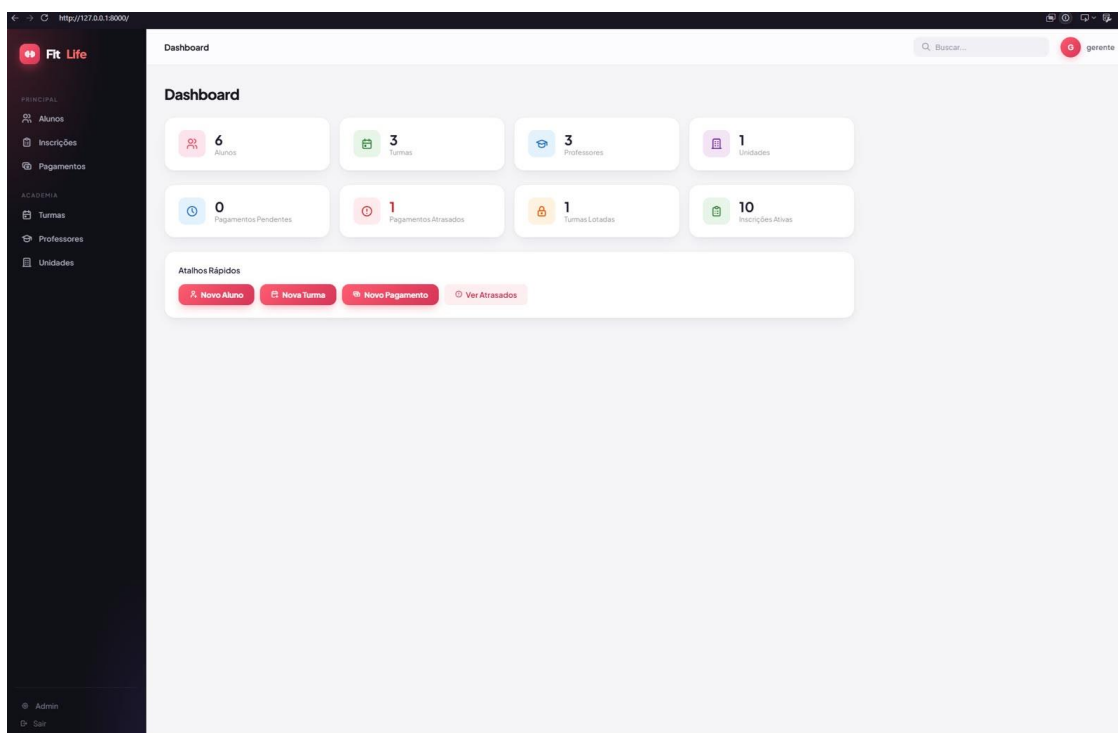


Figura 2: Tela de Dashboard (painel gerencial)

Fonte: Elaborado pelos autores (2026).

4.2 AUTENTICAÇÃO

4.2.1 Tela de Login

Objetivo: Controlar o acesso ao sistema, exigindo autenticação do usuário.

Descrição: Apresenta os campos para informar as credenciais de acesso e, após a validação, encaminha o usuário ao painel principal de acordo com seu perfil.

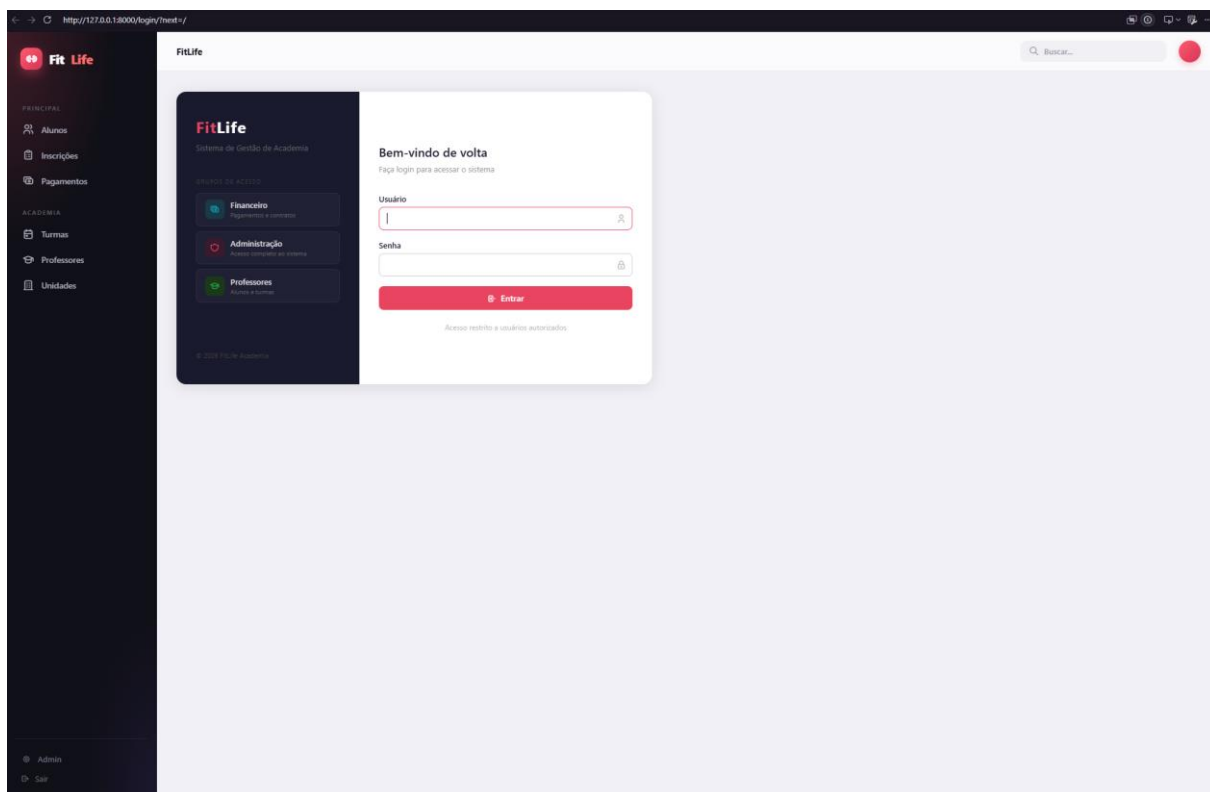


Figura 3: Tela de Login (acesso ao sistema)

Fonte: Elaborado pelos autores (2026).

4.3 MÓDULO DE ALUNOS

4.3.1 Listagem de Alunos

Objetivo: Listar os alunos cadastrados e permitir a busca, a visualização e a manutenção de seus registros.

Descrição: Exibe a relação de alunos com nome, CPF, telefone e e-mail, sinalizando eventuais restrições médicas por meio de etiquetas. Disponibiliza campo de busca por nome, ordenação e as ações Ver, Editar e Excluir para cada aluno, além do botão para cadastrar um novo aluno.

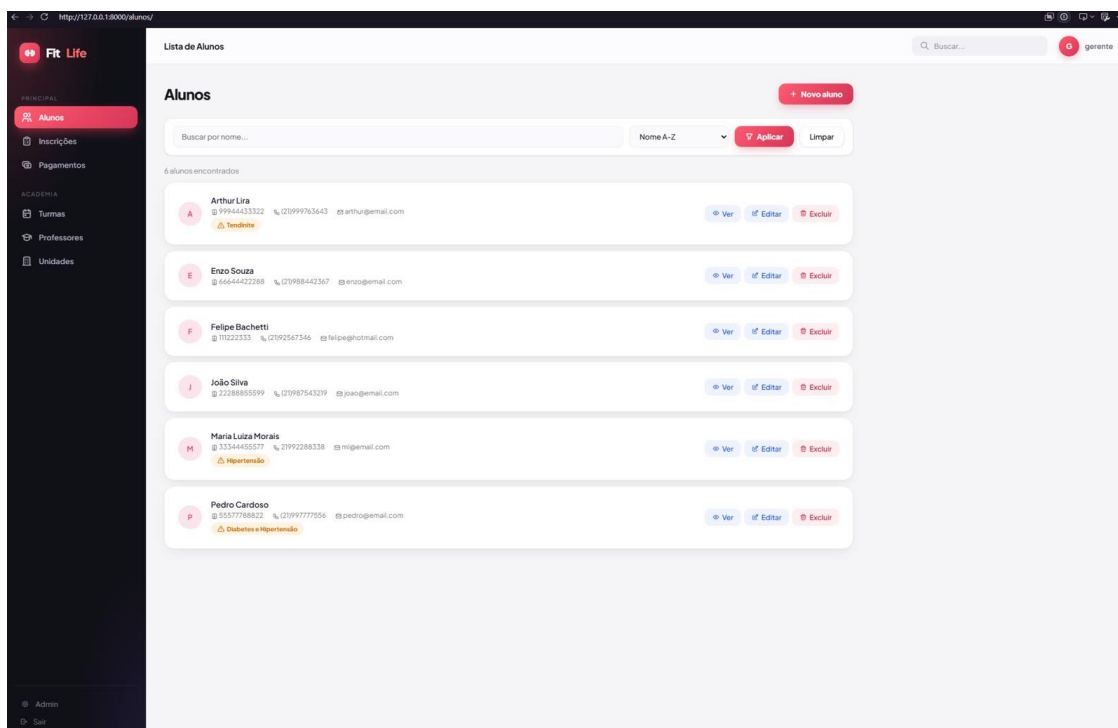


Figura 4: Tela de Listagem de Alunos

Fonte: Elaborado pelos autores (2026).

4.3.2 Cadastro de Aluno

Objetivo: Registrar um novo aluno no sistema.

Descrição: Formulário com os campos CPF, nome completo, data de nascimento, telefone, endereço, e-mail e restrições médicas (opcional). Os campos obrigatórios são destacados e há validação no envio dos dados.

The screenshot shows a web application interface for 'Fit Life'. On the left is a dark sidebar with a menu containing 'Alunos', 'Inscrições', 'Pagamentos', 'Turmas', 'Professores', and 'Unidades'. The main area is titled 'Novo Aluno' and contains a registration form with the following fields: 'CPF' (with a hint 'Somente números'), 'Nome Completo', 'Data de Nascimento' (mm/dd/yyyy), 'Telefone' (with a hint '(21) 99999-0000'), 'Endereço' (with a hint 'Rua, número, bairro, cidade'), 'E-mail' (with a hint 'aluno@email.com'), and 'Restrições Médicas' (with a hint 'Ex: hipertensão, problema no peito...'). At the bottom of the form are 'Salvar' and 'Cancelar' buttons. A 'Voltar' button is located at the top right of the form area. The top of the page includes a search bar and a user profile for 'gerente'.

Figura 5: Tela de Cadastro de Aluno

Fonte: Elaborado pelos autores (2026).

4.3.3 Detalhes do Aluno

Objetivo: Apresentar de forma consolidada as informações de um aluno.

Descrição: Exibe os dados cadastrais do aluno e destaca, em área específica, as restrições médicas registradas. Disponibiliza os botões para editar ou excluir o registro.

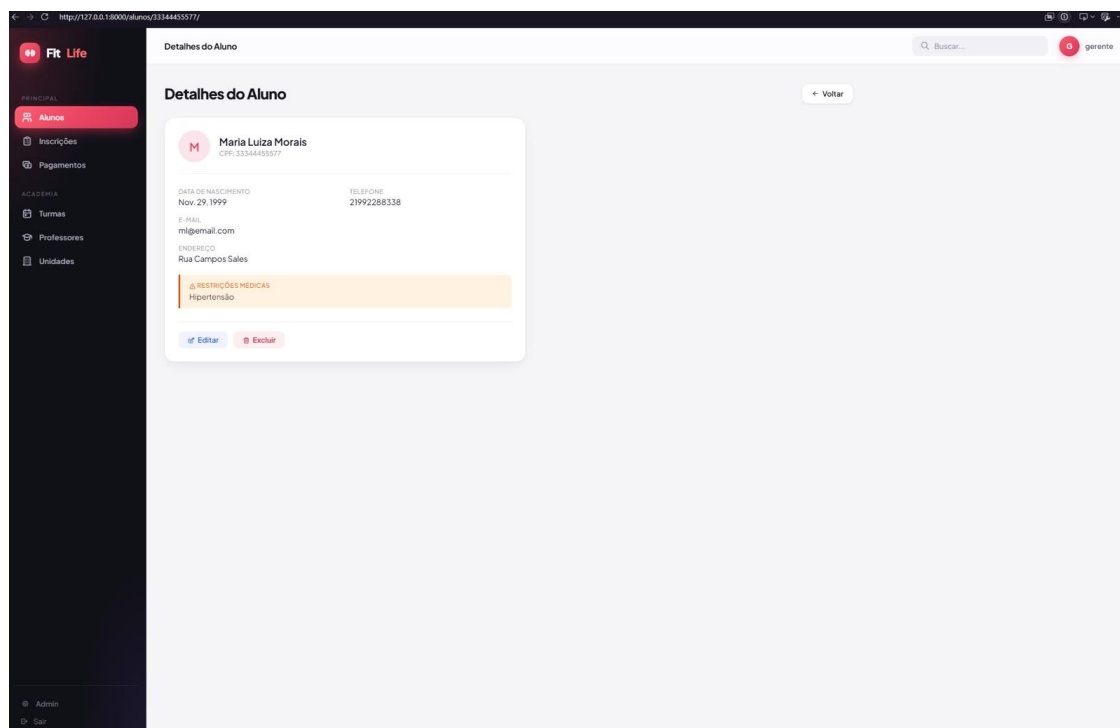


Figura 6: Tela de Detalhes do Aluno

Fonte: Elaborado pelos autores (2026).

4.3.4 Edição de Aluno

Objetivo: Alterar os dados de um aluno já cadastrado.

Descrição: Reutiliza o formulário de cadastro, carregando previamente as informações do aluno selecionado para atualização.

Fit Life

PRINCIPAL

- Alunos
- Inscrições
- Pagamentos

ACADEMIA

- Turmas
- Professores
- Unidades

Admin

Editar Aluno

CPF (obrigatório)

99944433322

Nome Completo *

Arthur Lira

Data de Nascimento *

07/12/1994

Telefone *

(21)999763643

Endereço *

Rua Cipriano Vieira

E-mail *

arthur@gmail.com

Restrições Médicas (opcional)

Tendinite

✓ Salvar alterações Cancelar

+ Voltar

Buscar...

gerente

Figura 7: Tela de Edição de Aluno
Fonte: Elaborado pelos autores (2026).

4.3.5 Exclusão de Aluno

Objetivo: Remover um aluno do sistema mediante confirmação.

Descrição: Apresenta uma tela de confirmação com os dados do aluno (CPF, e-mail e telefone) e alerta que a ação não poderá ser desfeita, evitando exclusões acidentais.

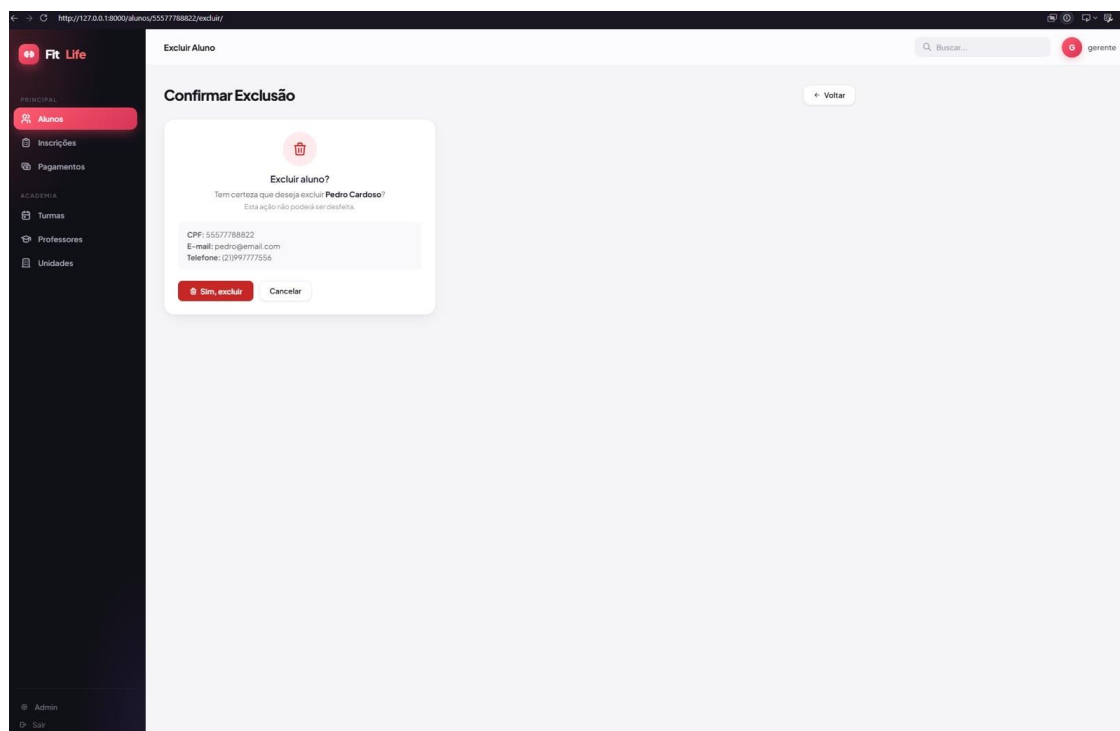


Figura 8: Tela de Confirmação de Exclusão de Aluno

Fonte: Elaborado pelos autores (2026).

4.4 MÓDULO DE TURMAS

4.4.1 Listagem de Turmas

Objetivo: Listar as turmas ofertadas e seu estado de ocupação.

Descrição: Relaciona as turmas com modalidade, professor responsável, unidade, nível de dificuldade e ocupação, sinalizando vagas disponíveis ou a condição de lotada. Oferece busca por modalidade, ordenação e as ações Ver e Editar.

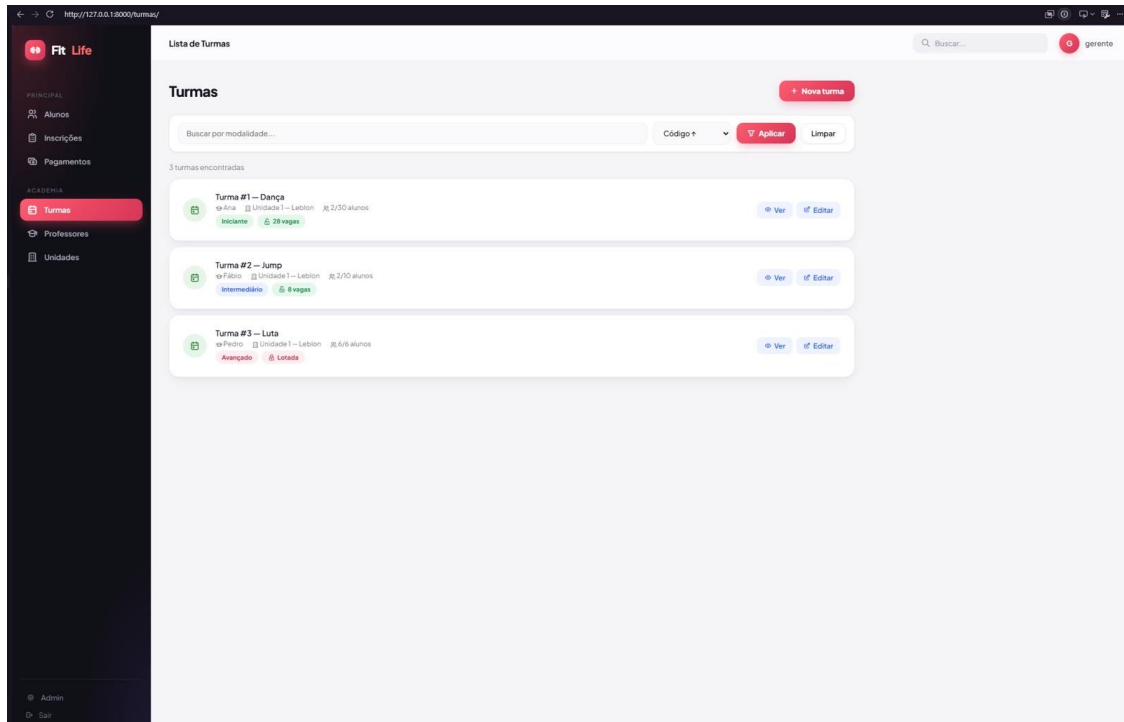


Figura 9: Tela de Listagem de Turmas

Fonte: Elaborado pelos autores (2026).

4.4.2 Cadastro de Turma

Objetivo: Criar uma nova turma.

Descrição: Formulário com seleção de modalidade, professor responsável, unidade e definição da capacidade máxima de alunos.

The screenshot shows a web application interface for 'Fit Life'. On the left is a dark sidebar with navigation links: 'PRINCIPAL', 'Alunos', 'Inscrições', 'Pagamentos', 'ACADEMIA', 'Turmas' (highlighted in red), 'Professores', and 'Unidades'. At the bottom of the sidebar are 'Admin' and 'Sair' options. The main content area is titled 'Nova Turma' and contains a form with the following fields: 'Modalidade *' (a dropdown menu with the placeholder 'Selecione uma modalidade...'), 'Professor Responsável *' (a dropdown menu with the placeholder 'Selecione...'), 'Unidade *' (a dropdown menu with the placeholder 'Selecione...'), and 'Capacidade Máxima *' (a text input field with the placeholder 'Ex.: 30'). Below the form are two buttons: 'Salvar' (with a checkmark icon) and 'Cancelar'. In the top right corner of the main area, there is a 'Voltar' button. The top of the page features a search bar with the placeholder 'Buscar...' and a user profile icon labeled 'gerente'.

Figura 10: Tela de Cadastro de Turma

Fonte: Elaborado pelos autores (2026).

4.4.3 Detalhes da Turma

Objetivo: Exibir as informações de uma turma e os alunos nela inscritos.

Descrição: Mostra modalidade, duração, dificuldade, capacidade, professor responsável e unidade, além da relação de alunos inscritos, com a opção de inscrever novos alunos ou cancelar inscrições existentes.

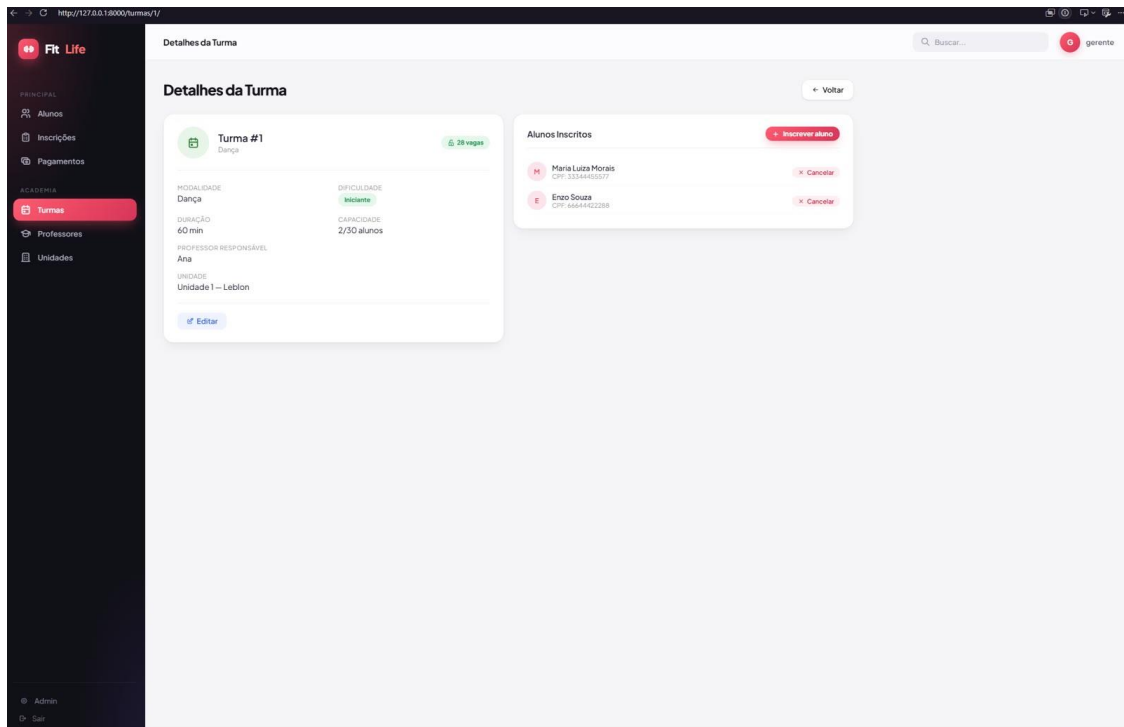


Figura 11: Tela de Detalhes da Turma

Fonte: Elaborado pelos autores (2026).

4.4.4 Edição de Turma

Objetivo: Alterar os dados de uma turma existente.

Descrição: Permite modificar modalidade, professor responsável, unidade e capacidade máxima, mantendo o código da turma como campo não editável.

Editar Turma

Código da Turma (não editável)

#1

Modalidade *

Dança (Iniciante - 60 min)

Professor Responsável *

Ana

Unidade *

Unidade1 - Leblon

Capacidade Máxima *

30

Salvar alterações Cancelar

Voltar

Figura 12: Tela de Edição de Turma

Fonte: Elaborado pelos autores (2026).

4.4.5 Inscrição de Aluno em Turma

Objetivo: Vincular um aluno a uma turma, respeitando a capacidade disponível.

Descrição: Exibe um resumo da turma (código, modalidade, professor e vagas) e permite selecionar o aluno a ser inscrito. A operação respeita o limite de vagas da turma.

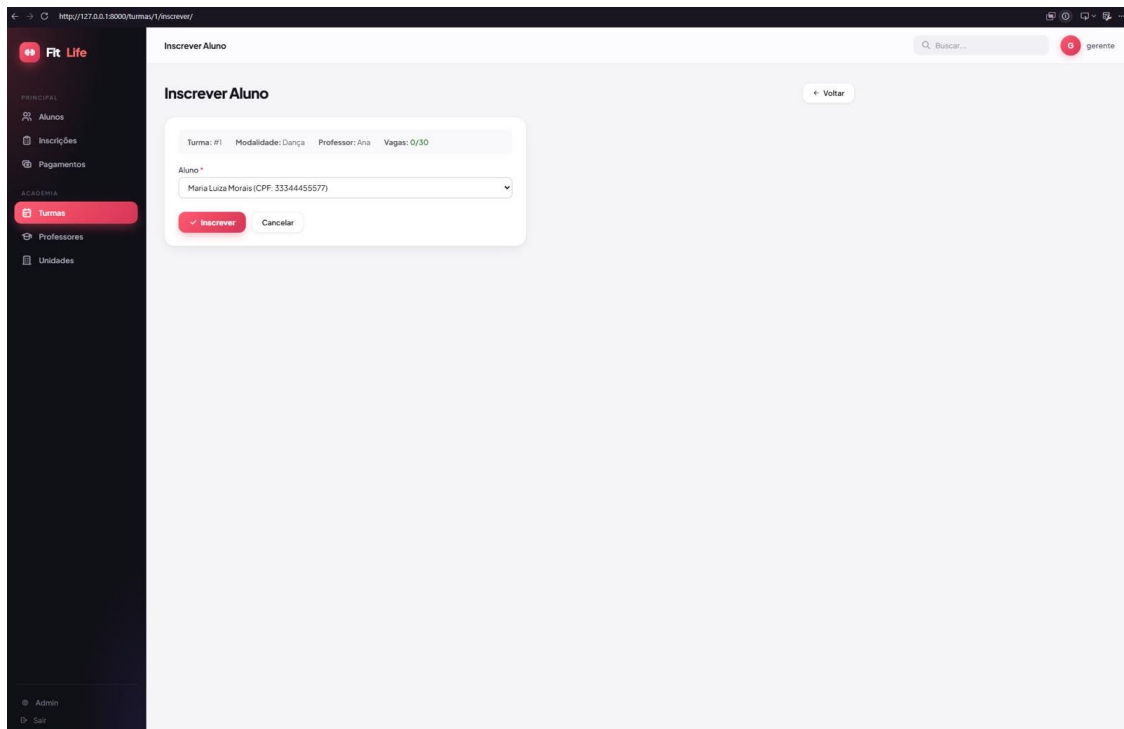


Figura 13: Tela de Inscrição de Aluno em Turma

Fonte: Elaborado pelos autores (2026).

4.5 MÓDULO DE INSCRIÇÕES

4.5.1 Listagem de Inscrições

Objetivo: Consolidar as inscrições ativas dos alunos nas turmas.

Descrição: Apresenta a relação de inscrições registradas no sistema, associando alunos às respectivas turmas.

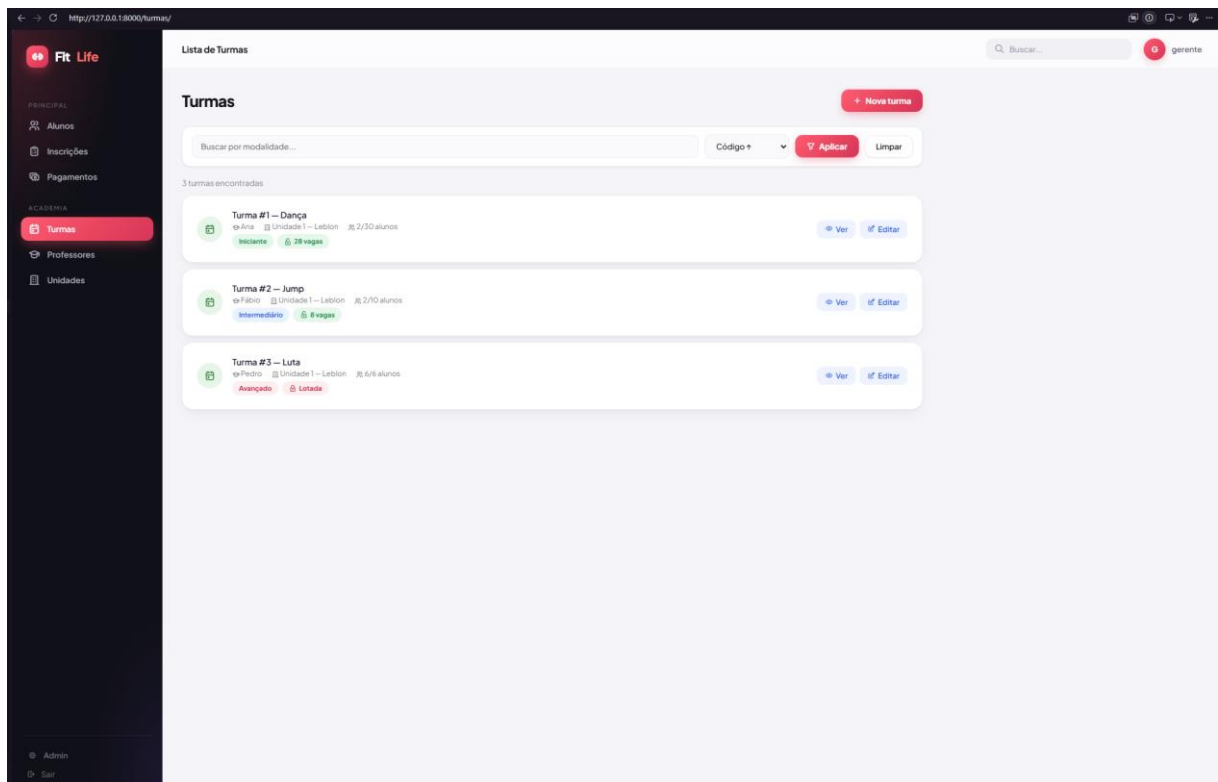


Figura 14: Tela de Listagem de Inscrições

Fonte: Elaborado pelos autores (2026).

4.6 MÓDULO DE PROFESSORES

4.6.1 Listagem de Professores

Objetivo: Listar o corpo docente da academia.

Descrição: Relaciona os professores com número do CREF, telefone e e-mail, indicando o vínculo com turmas e o supervisor responsável, quando houver. Oferece busca por nome e ordenação.

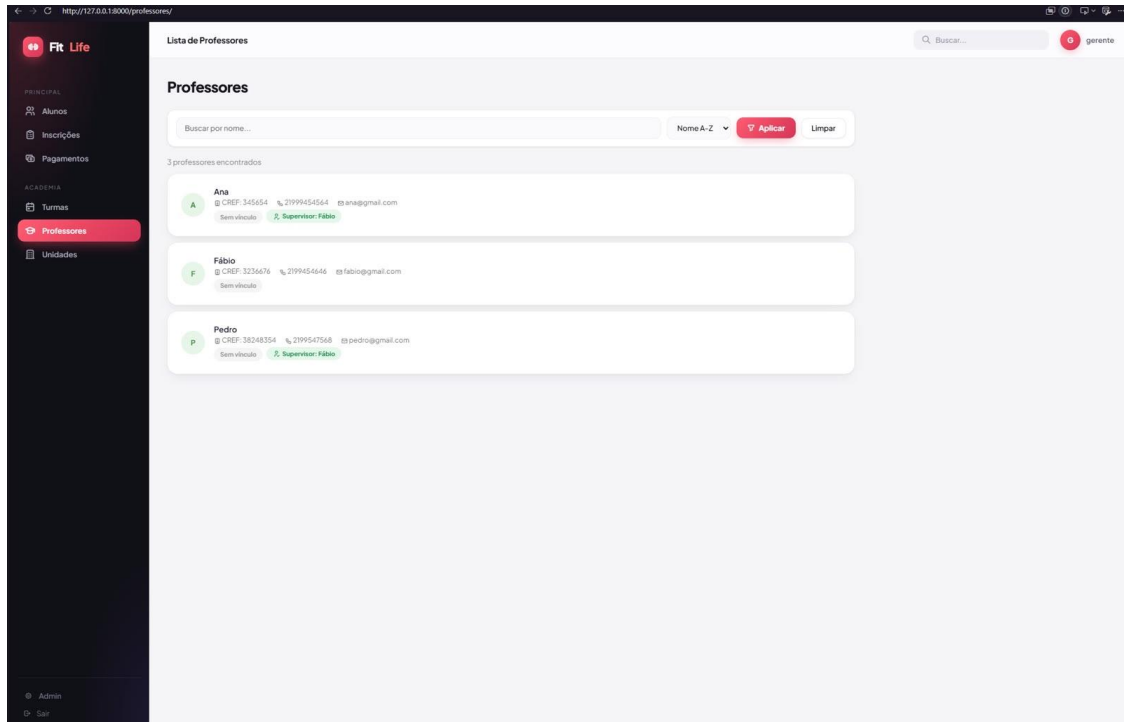


Figura 15: Tela de Listagem de Professores

Fonte: Elaborado pelos autores (2026).

4.7 MÓDULO DE UNIDADES

4.7.1 Listagem de Unidades

Objetivo: Listar as unidades (filiais) da rede.

Descrição: Exibe as unidades com endereço, telefone e horário de funcionamento, sinalizando a ausência de gerente responsável quando aplicável. Oferece busca por endereço e ordenação.

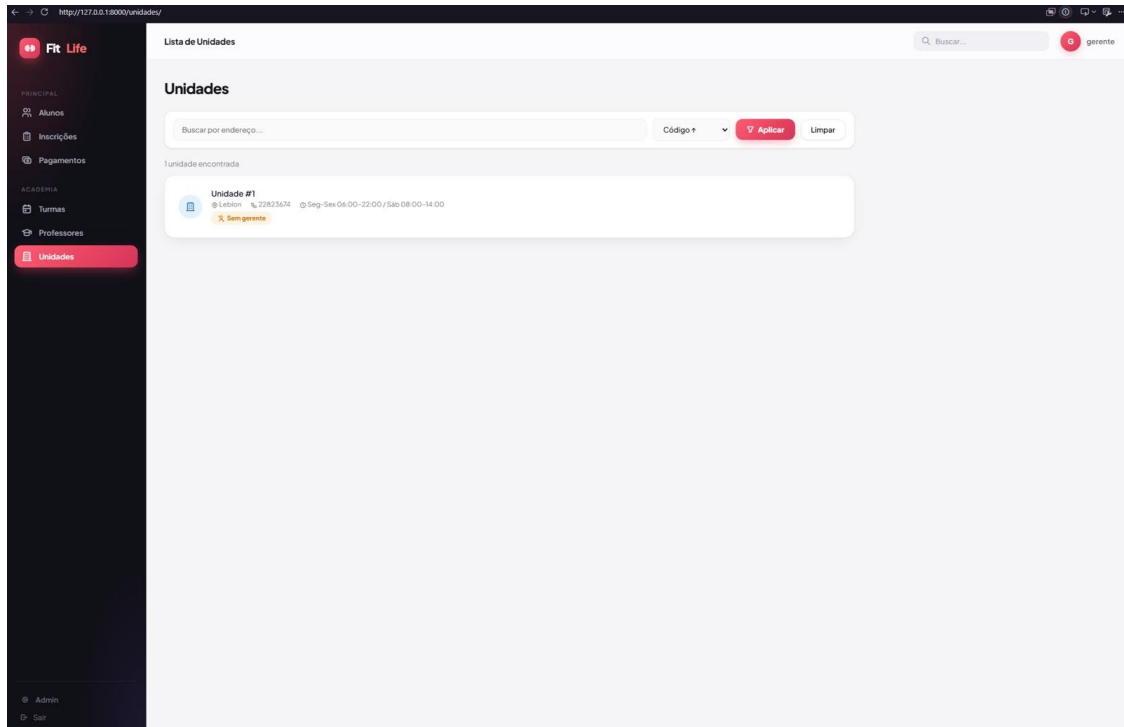


Figura 16: Tela de Listagem de Unidades

Fonte: Elaborado pelos autores (2026).

4.8 MÓDULO DE PAGAMENTOS

4.8.1 Listagem de Pagamentos

Objetivo: Acompanhar as cobranças e sua situação.

Descrição: Relaciona os pagamentos com o nome do aluno, contrato, forma de pagamento, vencimento, valor e status, destacando as cobranças atrasadas. Oferece busca por nome do aluno, filtro por status e ordenação por vencimento.

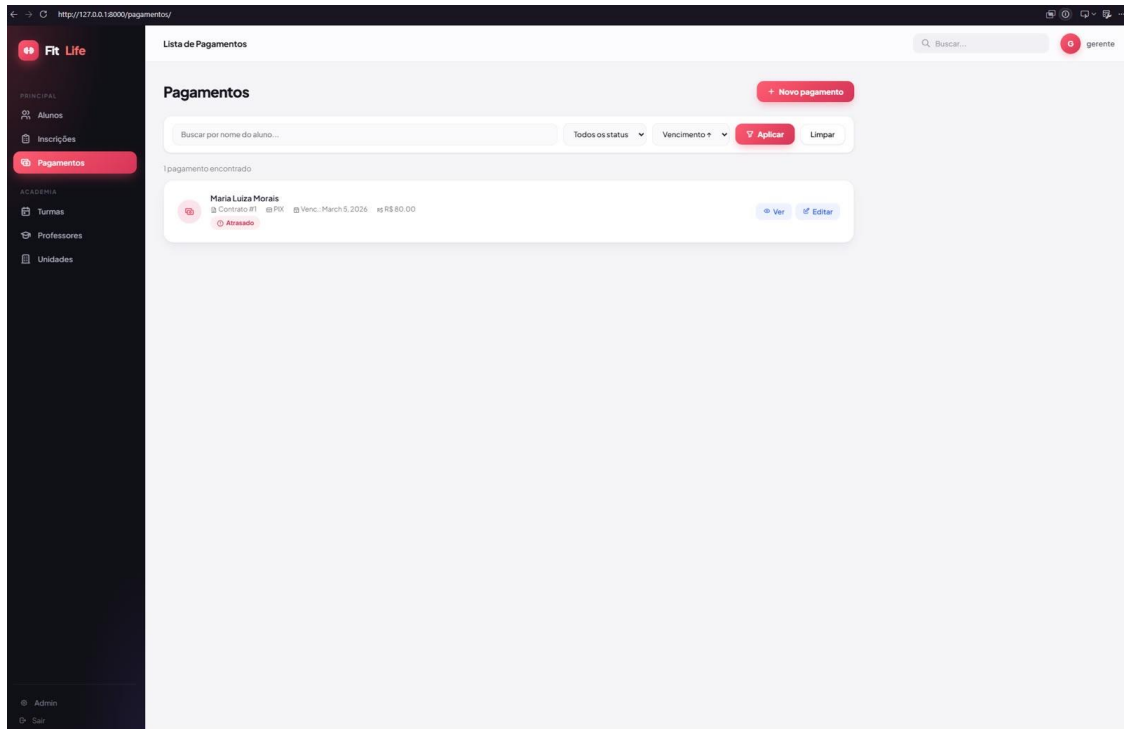


Figura 17: Tela de Listagem de Pagamentos

Fonte: Elaborado pelos autores (2026).

4.8.2 Cadastro de Pagamento

Objetivo: Registrar uma nova cobrança vinculada a um contrato.

Descrição: Formulário com seleção do contrato, forma de pagamento, status, valor e data de vencimento.

The image shows a web application interface for 'Fit Life'. On the left is a dark sidebar with the logo and a menu containing 'Alunos', 'Inscrições', 'Pagamentos' (highlighted), 'Turmas', 'Professores', and 'Unidades'. The main area displays a 'Novo Pagamento' form. The form has a title 'Novo Pagamento' and a 'Voltar' button. It contains several input fields: 'Contrato*' (a dropdown menu), 'Forma de Pagamento*' (a dropdown menu), 'Status*' (a dropdown menu with 'Pendente' selected), 'Valor (R\$) *' (a text input field with '0.00'), and 'Vencimento*' (a date input field with the format 'mm/dd/yyyy'). At the bottom of the form are two buttons: 'Salvar' (with a checkmark icon) and 'Cancelar'. The browser's address bar shows 'http://127.0.0.1:8000/pagamentos/cadastrar/'.

Figura 18: Tela de Cadastro de Pagamento

Fonte: Elaborado pelos autores (2026).

4.8.3 Detalhes do Pagamento

Objetivo: Exibir as informações completas de uma cobrança.

Descrição: Mostra valor, vencimento, forma de pagamento, contrato, aluno e plano associados, além do status atual do pagamento (por exemplo, atrasado).

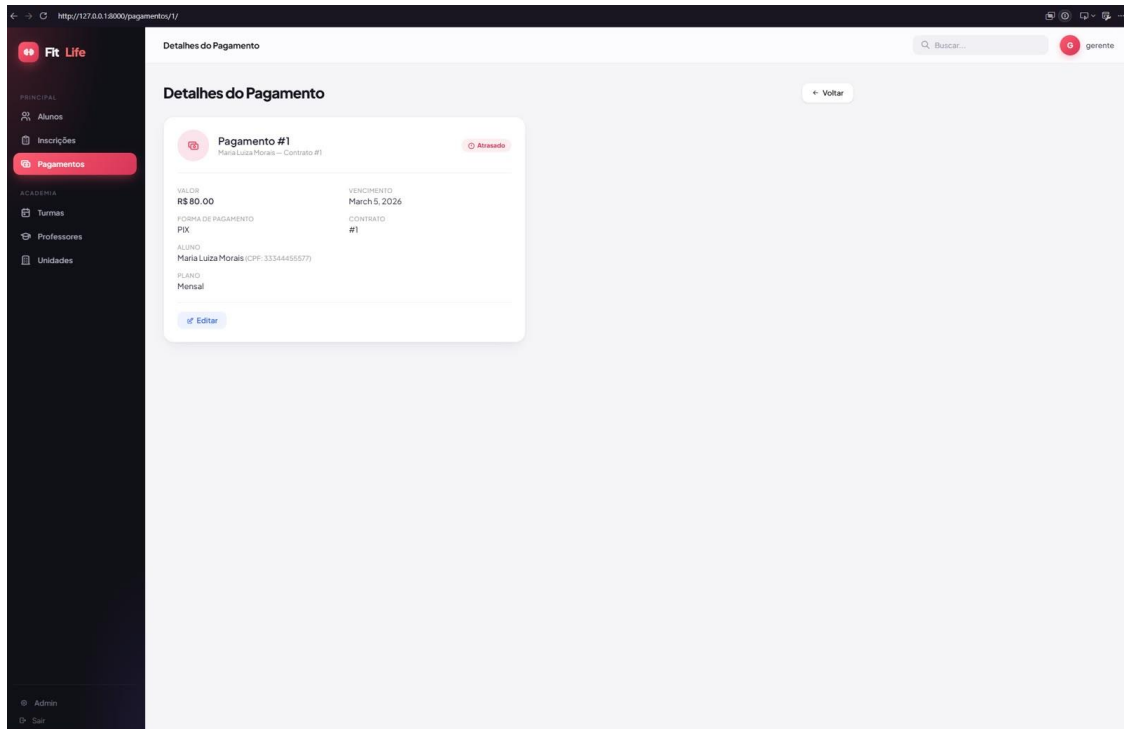


Figura 19: Tela de Detalhes do Pagamento

Fonte: Elaborado pelos autores (2026).

4.8.4 Edição de Pagamento

Objetivo: Atualizar os dados de uma cobrança existente.

Descrição: Permite alterar a forma de pagamento, o status, o valor e o vencimento, mantendo o código do pagamento e o contrato como campos não editáveis.

The screenshot shows a web application interface for editing a payment. The left sidebar contains navigation links: 'PRINCIPAL', 'Alunos', 'Inscrições', 'Pagamentos' (highlighted), 'ACADEMIA', 'Turmas', 'Professores', 'Unidades', 'Admin', and 'Sair'. The main content area is titled 'Editar Pagamento' and features a form with the following fields:

- Código do Pagamento** (Info editável): A text input field with a placeholder '#1'.
- Contrato** (Info editável): A text input field with the value 'Contrato #1 - Maria Luiza Morais'.
- Forma de Pagamento ***: A dropdown menu with 'PIX' selected.
- Status ***: A dropdown menu with 'Atrasado' selected.
- Valor (R\$) ***: A text input field with the value '80.00'.
- Vencimento ***: A date input field with the value '03/05/2026' and a calendar icon.

At the bottom of the form are two buttons: '✓ Salvar alterações' and 'Cancelar'. A 'Voltar' button is also visible in the top right corner of the form area.

Figura 20: Tela de Edição de Pagamento

Fonte: Elaborado pelos autores (2026).

5 ORGANIZAÇÃO ÁGIL E BACKLOG

O desenvolvimento do FitLife foi organizado de forma incremental, com o trabalho dividido em etapas encadeadas que evoluíram do planejamento até a entrega final. O acompanhamento das atividades foi realizado por meio de uma ferramenta de gestão de tarefas (GitHub Projects/Trello), na qual as demandas eram registradas, priorizadas e movimentadas entre as colunas de planejadas, em andamento e concluídas.

O quadro a seguir sintetiza o backlog do projeto, associando cada etapa às principais entregas e à mensagem de commit representativa, conforme o plano de desenvolvimento adotado.

Etapa	Principais entregas	Commit representativo
1	Definição do sistema, perfis de usuário, tecnologias e criação do repositório.	chore: estrutura inicial do projeto
2	Modelagem das entidades, DER e script de criação das tabelas.	feat: modelagem inicial do banco de dados
3	Configuração do ambiente, requirements.txt, conexão com o banco e autenticação.	feat: configuração inicial do projeto e autenticação
4	Models e regras de negócio: alunos, planos, pagamentos e vencimentos.	feat: model de pagamentos e regra de vencimento
5	Views, validações, camada de serviços, filtros e buscas.	feat: views de cadastro de aluno e listagem
6	Templates e telas: dashboard, alunos, turmas, pagamentos e demais módulos.	feat: template do dashboard e tela de alunos
7	Testes manuais, verificação de perfis e correções.	fix: correção de validação no cadastro de aluno
8	Elaboração da documentação acadêmica.	docs: documentação acadêmica do projeto
9	Escrita do README e organização final do repositório.	docs: README e organização do repositório
10	Preparação e envio dos materiais de entrega.	chore: preparação da entrega final

6 VERSIONAMENTO E GITHUB

O código-fonte do projeto foi versionado com Git e hospedado no GitHub, garantindo o histórico das alterações e o trabalho colaborativo entre os integrantes da equipe. As contribuições foram registradas por meio de commits com mensagens padronizadas, segundo prefixos semânticos que identificam a natureza de cada alteração.

Link do repositório: <https://github.com/ruanhilariorj1/desenvolvimento-de-software->

Os exemplos a seguir ilustram o padrão de mensagens de commit utilizado ao longo do desenvolvimento:

```
feat: criação da tela de login
fix: correção da validação de usuário
refactor: reorganização da camada de services
```

A estrutura de pastas do repositório foi organizada de modo a separar o código-fonte, os arquivos de documentação e os recursos estáticos, conforme apresentado a seguir.

fitlife-gestao-de-academia/

```
|— config/      → configurações globais, urls principal, wsgi
|— fitlife/     → app principal
|  |— models.py → entidades e ORM
|  |— views.py  → lógica de negócio e controle de acesso
|  |— urls.py   → roteamento de URLs
|  |— admin.py  → registro no painel administrativo
|  └— templates/
|     └— fitlife/ → templates HTML da interface
|— templates/
|  └— registration/
|     └— login.html
|— docs/        → documentação técnica
|  └— telas/
|— manage.py
└—
```

requirements.txt

7 RESULTADOS

Ao término do desenvolvimento, obteve-se uma aplicação web funcional que atende ao escopo definido para a gestão administrativa de uma academia. O sistema permite realizar as operações de criação, leitura, edição e exclusão sobre todas as entidades principais do domínio — alunos, turmas, professores, unidades e pagamentos — e implementa fluxos de negócio específicos, com destaque para a inscrição de alunos em turmas com controle de vagas e o acompanhamento de pagamentos por situação.

O painel gerencial consolidou, em uma única tela, os principais indicadores da operação, apresentando os totais de alunos, turmas, professores e unidades cadastrados, bem como o número de pagamentos pendentes e atrasados, de turmas lotadas e de inscrições ativas. Esse recurso confere à gerência uma visão rápida da situação do negócio e facilita a identificação de pendências, como cobranças em atraso e turmas sem vagas.

Os testes manuais realizados ao longo do projeto verificaram o comportamento das validações, a navegação entre as telas e a consistência das regras de negócio, contribuindo para a estabilidade da versão entregue. As capturas apresentadas no Capítulo 4 evidenciam o funcionamento das principais funcionalidades implementadas.

8 CONCLUSÃO

Este trabalho apresentou o desenvolvimento do FitLife, um sistema web para a gestão de academias, concebido a partir de um estudo de caso do domínio. O projeto percorreu todas as etapas de um ciclo de desenvolvimento de software, desde o planejamento e a modelagem do banco de dados até a implementação das funcionalidades, os testes e a documentação, atingindo os objetivos propostos.

A construção da solução permitiu articular, de forma integrada, conhecimentos de modelagem de dados, arquitetura de software em camadas, desenvolvimento web com o framework Django e boas práticas de versionamento com Git e GitHub. A organização do trabalho em etapas incrementais mostrou-se adequada, favorecendo o acompanhamento do progresso e a divisão de responsabilidades entre os integrantes da equipe.

Como perspectivas de trabalhos futuros, destacam-se a implementação de perfis de acesso diferenciados para recepcionistas e alunos, o registro de frequência e de avaliações físicas descritos no estudo de caso, a geração de relatórios financeiros mais completos e a disponibilização de uma API REST para integração com um aplicativo móvel. Tais evoluções ampliariam o alcance da solução e aproximariam o sistema das necessidades reais de uma rede de academias.

Link do GitHub: <https://github.com/ruanhilarioj1/fitlife-gestao-de-academia>

REFERÊNCIAS

DJANGO SOFTWARE FOUNDATION. Django documentation. Disponível em: <https://docs.djangoproject.com>. Acesso em: 3 jul. 2026.

PYTHON SOFTWARE FOUNDATION. Python 3 documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 3 jul. 2026.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2018.

CHACON, S.; STRAUB, B. Pro Git. 2. ed. Apress, 2014. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 3 jul. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.